LANGUAGE MACHINE

LatinCy Workshop Feb 2026

Project Lingua ex machina / Language Machine

An evolving early stage project aiming to develop an offline, open-source generative AI platform to support under-represented languages within the humanities.

Designed to run on ordinary consumer hardware without cloud services or complex software infrastructure, the system will enable search, textual analysis, and generative workflows over locally hosted corpora while preserving scholarly control over data.

Adam Gitner <agitner@thesaurus.badw.de >

Edward C. Zimmermann <edz@nonmonotonic.net>

A WARNING

I am **not** a philologist. While my kids have German Latinum and Graecum I know neither. I assume I'll be making some bold erroneous statements about Latin but they should be taken for that they are: cargo-cultish observations.

The project is a product of many discussions I've had with Adam (Gitner) over the years.

My own work bridges commercial consulting and public research, with decades of experience focused on AI/ML, NLP, and information retrieval.

Many of my projects have been supported by the EU Next Generation Internet (NGI); German Federal Ministry for Research, Technology and Space; and an assortment of Civil Foundations.

I also contribute actively to international IT standards bodies, including the European Observatory for ICT Standardization, and teach intensive courses to researchers (mainly in Europe).

Edward Zimmermann

Data Scientist/AI/Computer Vision Researcher

https://www.linkedin.com/in/edwardzimmermann

The Motivation

Contemporary AI systems largely exclude historical and low-resource languages, forcing scholars to rely on commercial platforms optimised for modern, high-resource vernaculars. They typically require users to upload data to remote cloud services. Such architectures create structural barriers for minority language communities, cultural heritage institutions, educators, and public-sector organisations that require secure, long-term, locally controlled tools rather than commercial platforms built around data extraction and centralised infrastructure.

Most models tend to fall to the “Beige Problem”: LLMs view text as a kind of "statistical average" of their training data. They operate through syntactic processing rather than conceptual understanding. While they can show an almost “human-like” ability to understand context and nuance in text in general-purpose, conversational, or high-resource language scenarios they come short in literature and prose. Handling of nuance is inconsistent. They often struggle with deep, culturally specific, or highly specialized subtleties.

Our project addresses this gap by aligning language technology with humanistic research practices and European priorities of linguistic diversity and digital sovereignty.

The Goal: run locally on consumer hardware

This project sets out to develop an offline, open-source generative AI platform designed to run on standard consumer hardware and to be adaptable to any language. It will integrate neural code generation with word-based (lexical) and semantic (vector) search over curated, locally hosted corpora, thereby enabling search, document analysis, and generative text workflows without reliance on external servers or specialised infrastructure. The system is designed to require neither Docker nor Python, and avoid complex software dependencies. The code-base is primarily C++ (although we will offer bindings to other languages such as Python).

The primary objective is to deliver an accessible, privacy-respecting solution for underserved and minority languages across Europe. By doing so, the project contributes to linguistic diversity, digital sovereignty, and equitable access to advanced language technologies. As an initial testbed, we focus on Latin and Ancient Greek.

Latin is not the goal but an interesting means

Latin serves as the primary testbed, not as a legacy language but as a linguistically demanding case study. Its rich morphology, complex syntax, and well-structured textual traditions allow us to evaluate NLP methods under conditions that resist simplistic statistical modelling.

Latin and Ancient Greek provide extensive open-access corpora, exhibit highly complex grammatical structures that place significant stress on NLP systems, and are supported by active digital humanities communities capable of offering informed and structured feedback. These properties make them well suited for validating our approach prior to scaling to genuinely underserved and endangered European languages.

Why is Classical Latin especially difficult?

- LLMs learn from massive text corpora, but but many available Latin texts online are **medieval, ecclesiastical, or neo-Latin**, not Classical. Classical Latin literature represents a tiny fraction of the total corpus, estimated at roughly 0.01% of the total. (no more than 9 to 10 million words).
- Thesaurus Linguae Latinae (TLL) covers much more than just Classical Latin (aka Ancient Latin) —and for this we'll have to wait a few years (Adam can chime in here).
- Latin is a highly inflected language (nouns have case, number, gender and verbs encode tense, voice, mood, person, number) with a flexible word order (hyperbaton). While this provides stylistic richness, it creates significant parsing, grammatical ambiguity, and syntactical reasoning issues for LLMs.
- LLMs don't explicitly parse morphology—they predict tokens probabilistically. Their morphological awareness is generally implicit and comparatively limited (and inconsistent).

Why is Classical Latin especially difficult?

- Word order matters significantly to Large Language Models (LLMs) like GPT because they are fundamentally autoregressive, meaning they process and generate information sequentially, one token at a time, usually from left to right. This structure means the sequence in which words or instructions are presented directly influences the model's "attention" mechanism, semantic understanding, and reasoning ability. While word order matters in Classical Latin, its function is different.

- For semantic search (a task that differs from text generation) using a bidirectional BERT (Bidirectional Encoder Representations from Transformers) model with text provides significant advantages over traditional, unidirectional models by allowing the model to analyze a word based on both its preceding and following context simultaneously. This is particularly valuable for highly inflected languages like Latin, where word order is flexible, and semantic meaning is heavily dependent on context.

Why is Latin especially difficult?

- Latin text often differs drastically in style, vocabulary, and grammar from the training data of general-purpose LLMs (which are trained on modern internet text). This "domain mismatch" leads to poor performance on specialized tasks, such as translating archaic or highly poetic texts.
- Latin evolved over centuries (will talk about semantic drift later). The neo-Latin (approx. 1400–1800+) revival of Classical Latin (1st c. BC–1st c. AD) by Renaissance humanists tried to adhere to Classical grammatical rules and structures (syntax) in imitation of the style of Cicero and Caesar but with semantic shifts and a greatly expanded vocabulary—including many new terms for inventions, scientific discoveries, and modern concepts. Despite attempts to stay pure, Neo-Latin was sometimes influenced by the speaker's native tongue (e.g., German, Italian, or French), leading to minor semantic or stylistic shifts in usage.
- Despite being a well-studied language, Latin is considered low-resource in AI terms because the sheer volume of digitized, parallel text is far lower than languages like English, French, German or Spanish. This leads to lower performance in zero-shot or few-shot learning scenarios compared to high-resource languages.
- There is a lack of automated, robust, and nuanced evaluation benchmarks specifically designed for historical, low-resource languages. Standard, quantitative metrics (like BLEU) often fail to capture the semantic and aesthetic quality of a translation.

Hyperbation Issues

- Hyperbaton, the rhetorical technique of disrupting normal word order (rearranging sentences, like "This I must see" instead of "I must see this"), is a powerful tool for emphasis and poetic effect, but it carries several significant pitfalls.
    - Degraded Semantic Understanding: LLMs often rely on fixed, standard patterns (such as Subject-Verb-Object in English). When presented with inversions, the model might struggle to correctly identify the subject or object, leading to a "Potemkin" effect where the text appears fluent but is internally nonsensical.
    - Increased Hallucination Risk: Because hyperbaton disrupts the standard pattern, the model is more likely to "guess" the meaning, resulting in higher rates of hallucination—generating false or irrelevant information while maintaining a confident tone.
    - Failed Logical Reasoning: In complex tasks requiring logic, inversions can disrupt the attention mechanism. If an LLM is asked to reason through a problem structured with heavy hyperbaton, its ability to follow long-range dependencies breaks down, causing it to fail on multi-step tasks.
    - Poor Context Retention: LLMs process input as token sequences and may misinterpret the relationships between distant words when normal grammar is broken. This leads to a loss of coherence, especially in long-form narratives or nuanced arguments.

In essence, while LLMs can mimic archaic or poetic language (hyperbaton), they lack the true linguistic understanding to process it reliably, often treating it as a pattern to be imitated rather than a structure to be parsed.

“LLMs are optimized for high-volume, modern, well-represented languages. Latin is the opposite: low-resource, highly structured, stylistically constrained, and historically layered.”

What other languages pose these difficulties?

- There are many low resource languages including many minority languages in Europe.
- There are many more highly Inflected / Morphologically Rich Languages. Example: Finnish, Hungarian, Lithuanian, Georgian. A single verb can have hundreds of forms, so each form appears rarely in training data.
- Free or flexible word order languages: example Russian, Polish, Turkish.
- Diglossic Languages. These exist in multiple parallel forms. Examples: Arabic (Modern standard + many dialects), Hindi versus Urdu. Norwegian (Bokmål vs. Nynorsk).
- Encoding issues (why Unicode is a poor model). Script or orthography complicates tokenization and training. This is particularly problematic with Khmer, Thai, Tibetan as there are no hard spaces between words or complex glyph combinations.
- The most extremely challenging languages are those that combine *multiple difficulty factors at once*: scarce corpora + rich morphology + inconsistent orthography. Examples: Indigenous American languages, many African languages with limited digitization (e.g., Wolof in Senegal, Tigrinya in Eritrea), Caucasian languages (e.g., Georgian, Chechen), Inuit languages (e.g., Inuktitut).
- Within Europe: Finnish, Hungarian, Lithuanian and Basque are highly challenging as despite decent data their grammar explodes the possible word forms making statistical learning inefficient.

Orthographic and grammatical Interference

- Here are also examples of languages with legislated grammatical or orthographic changes:
  - German (1901, 1996): The 1901 reform standardized spelling nationwide. The 1996 spelling reform (reform of German orthography) was a major, legally-backed change that, while primarily orthographic, affected grammar rules regarding capitalization, hyphenation, and word separation.
  - Dutch (1883, 1934, 1947, 1996, 2006): Through the Dutch Language Union (Taalunie), the Netherlands and Belgium have periodically updated their language. 20th-century reforms removed some grammatical case endings and simplified the spelling of words (e.g., changing de Nederlandsche tael to de Nederlandse taal).
  - French (1740, 1835, 1992): The Académie Française has periodically updated the grammar and spelling of French, which are then authorized for use in official documents and education.
  - Russian (1918): Following the revolution, a major, state-mandated reform abolished several letters (ѣ, і, ѳ, ѵ) and simplified some grammatical endings (e.g., in adjectives and pronouns), marking a significant shift in written, and to some extent spoken, Russian.
  - Turkish (1928): As part of Atatürk's reforms, the government abolished the Arabic script in favor of a new Latin-based alphabet and intentionally dropped many Persian and Arabic loanwords, heavily altering the grammar and vocabulary toward a more "native" Turkish structure.
  - Chinese (1950s–Present): The PRC government introduced Simplified Chinese characters, which was later adopted by Singapore and Malaysia, legally changing the standard written form. As of 2026, China continues to amend its Common Language Law to standardize and expand the use of Mandarin.
  - Norwegian (1907, 1917, 1938): Following independence from Denmark, Norway implemented reforms to make the written language (Bokmål) closer to spoken Norwegian, modifying grammar and spelling, and attempting to bridge it with Nynorsk.
  - Portuguese (20th Century): Multiple reforms, most notably the 1990 Portuguese Language Orthographic Agreement (implemented later), standardized spelling across Brazil and Portugal, simplifying phonetic spelling (e.g., dropping silent letters).
  - Irish (1940s): The government simplified the spelling system, removing silent letters to make the language more accessible.
  - Somali (1972): The government of Somalia mandated a new, standardized Latin script to replace the multiple, unofficial writing systems in use. established the Latin script as the official orthography, transforming a formerly oral language into a written, standardized administrative and educational medium. Driven by the Siad Barre government to increase literacy (aiming for 100% in rural areas) and remove "colonization of the brain," it replaced English, Italian, and Arabic in public life.

The Deliverable

Build upon Project שמאטע (Schmate pronounced “SHMAH-teh which means rag in Yiddish) to create a highly flexible generative AI system to construct sophisticated hybrid searches directly from natural-language queries recognising that most users are not trained information specialists.

The intent is to create a ready-to-use “out of the box” solution.

This entails a computationally constrained future forward building block that can be integrated within other frameworks, used with re-Isearch but also used by itself. Specific to re-Isearch is a query code generator akin to a “Github co-pilot” for the re-Isearch query language but also for the OASIS Contextual Query Language (CQL) which is in widespread use in bibliographic catalogs and museum collection information systems.

It should be noted here that re-Isearch supports CQL queries as well as interfaces to the ISO 23950:1998 (aka Z.39.50) protocol which continues to be widely used by national libraries including the US Library of Congress— which restored access last month (Jan) after a temporary outage that resulted from their migration to the FOLIO platform (which as standard builds on CQL via SRU/W).

Our שמאטע design addresses, beyond computational constraints, several well-documented limitations of contemporary retrieval-augmented generation (RAG) models, including semantic collapse, the statistical limits of dense retrieval at scale, and misaligned chunking strategies.

Central to our approach is a focus on textual structure and variable retrieval granularity, enabling AI-assisted inquiry that respects the organisation, semantics, and interpretive conventions of Latin texts, rather than reducing them to arbitrary fragments.

The Schmate Design: Be Powerful, Flexible and Efficient

Its design focus is for consumer grade hardware and edge computational devices (including a number of SOCs such as Qualcomm Snapdragon and NVIDIA Jetson platforms).

> ***“It's one thing to have a good re-search tool but it must also be economically feasible, efficient and sustainable.”***

Key needs:

- To create a highly flexible, computationally constrained future forward framework.
- 100% Open Source. Apache 2.0 License
- A model to support additional algorithms for Approximate Nearest Neighbour (ANN) search as use-cases arise.
- To address several well-documented limitations of contemporary retrieval-augmented generation (RAG) models, including semantic collapse—the statistical limits of dense retrieval at scale—and misaligned chunking strategies.
- High-performance on commodity hardware including x86_64, ARM, Apple Silicon (Metal), CUDA, etc.

Schmate delivers: Power, Flexibility and Efficiency

==>

- Semantic search is driven by Sentence-Transformers (sBERT) embeddings.
- Support of sBERT is Implemented in C++ leveraging the GGML tensor library via *llama.cpp* and *bert.cpp* to enable efficient on-device machine-learning inference. GGML's emphasis on high-performance tensor operations on commodity hardware is aligned with our central design philosophy.
- Our support of GGUF (GPT-Generated Unified Format) for LLMs opens the doors to many thousands of models from platforms like HuggingFace. While other formats exist (like Safetensors for GPU/general use or raw PyTorch) most of these can be converted to GGUF.
- We elected for "easy" integration of "current" and future models to support both GGUF and the older GGML model formats.
- For dense vector retrieval we start off with a turbo-charged and significantly enhanced implementation of HNSW (forking the popular HNSWlib C++ code)
- HNSW algorithms tend to outcompete other ANN algorithms in practical terms (providing sufficient memory is available) and the HNSWlib was already one of the fastest implementations.

For LLM we use, of course, the GGML Library

We use ggml via llama.cpp. Written in C++, it's optimized for speed.

It is highly flexible: You can fine-tune all settings including memory management, thread count, and GPU allocation.

Via ggml it has a very broad Hardware Support: Compatible with CPU, NVIDIA GPU, Apple Silicon (M1/M2/M3), and more.

Why not use Ollama?

Why not Ollama? Does it not build on llama.cpp (and, in turn, on the ggml tensor library) ?

The abstraction and network layer adds overhead. While not noticeable for simple queries, the difference becomes apparent for large-scale processing. llama.cpp is13%~80% faster than Ollama.

Ollama is NOT written in C++ (llama.cpp and ggml are in C++) but Ollama is written in Go using CGO to get Go bindings from llama.cpp and ggml.

Ollama connects to and listens on a specific network port (typically 11434), which allows it to function as a server and interact with CLI tools, web UIs, and applications.

Using sockets to connect to a local port is slower than direct in-memory function calls because it still utilises the operating system's network stack. Since many processes are I/O constrained this results alongside with the binding abstraction for the 13%-80% loss of performance.

Ollama also (as a ready to use system) has a few quirks such as, by default, unloading models from memory after 5 minutes of inactivity.

And more importantly: Ollama does not provide anything we need we don't get from llama.cpp! On the contrary it would get in the way.

Hybrid Search: At its core is Re-Isearch

Re-Isearch is a 100% open source (Apache 2.0) novel multimodal search and retrieval engine using mathematical models and algorithms different from the all-too-common inverted index. It is a kind of hybrid between full-text, XML, object and graph noSQL-db that natively ingests a wide range of document types and formats.

A further departure from conventional search systems lies in our approach to retrieval granularity. Traditional text indexers treat the document or record as both the unit of indexing and retrieval. In contrast, our system supports a dynamic, query-time unit of retrieval that may be specified by the user or inferred heuristically. Document structure is explicitly exploited to identify the most relevant components —such as chapters, pages, paragraphs, or lines—allowing retrieval at sub-document or supra-document levels as appropriate.

See my talk from FOSDEM '22: A lightning intro to re-Isearch.

https://archive.fosdem.org/2022/schedule/event/lt_re_Isearch/

re-Isearch was funded through the NGI0 Discovery Fund, a fund established by NLnet with financial support from the European Commission's Next Generation Internet programme, under the aegis of DG Communications Networks, Content and Technology under grant agreement N° 825322.

Portions also received funding from: Federal Ministry of Research, Technology and Space; OpenData CH/Mercator Foundation CH, EU StandICT. Earlier portions, among others, the US NSF.

(Re-)Isearch History

The past 30 years

- Isearch was a legendary open-source text retrieval software first developed in 1994 as part of the Isite Z39.50 information framework with support from NSF.
- Development was divided between CNDIR/MCNC (North Carolina, USA) and Bsn (Munich, Germany).
- In 1998 BSn launched a proprietary fork called IB using new algorithms that significantly improved performance and power.
- IB development shifted to BSn’s research arm: NONMONOTONIC Lab. In was deployed in 100s of high profile sites though BSn projects as well as partners.
- In 2011 Bsn/NONMONOTONIC's proprietary fork ceased with Ed Zimmermann leaving his active role in BSn and joining CIB.
- The software moved to the proverbial attic.
- Even ten years on from End-of-Development and despite lack of support some servers were still running!??? Don't break a working system.....

Who used it? Among others…

The U.S. Patent and Trademark Office (USPTO) patent search, the Federal Geographic Data Clearinghouse (FGDC), the NASA Global Change Master Directory, the NASA EOS Guide System, the NASA Catalog Interoperability Project, the astronomical pre-print service based at the Space Telescope Science Institute, The PCT Electronic Gazette at the World Intellectual Property Organization (WIPO), the SAGE Project of the Special Collections Department at Emory University, Eco Companion Australasia (an environmental geospatial resources catalog), the Open Directory Project, genomic search for the Australian National Genomic Information Service's (ANGIS) human genome project (and its eBiotechnology workbench split-off); the D-A-S-H search portal against racism, antisemitism and exclusion (funded within the framework of the action program "Youth for tolerance and democracy - against right-wing extremism, xenophobia and anti-Semitism", the YOUTH program of the European Community and with additional support from the German Federal Agency for Civic Education); the e-government search (Yeehaw) of the U.S. State of Utah to agronomic cooperation across the Mediterranean region (supported by the EU’s DG). Integrated into a number of CMS platforms it powered search for a number of high volume web sites. It was also used as a database accelerator by a number of eCommerce shops.

Re-Isearch. Its algorithms

Reborn in 2020 in the middle of the global Covid19 pandemic as Project re-Isearch catalysed by a grant from NInet/NGI-Zero. Like the original it is not just about textual words but pushes the envelope.

- Based roughly on pat-arrays going back to the original work of Gonnet, Baeza-Yates, Uri Mamber et al.
- The algorithm is accelerated by a simple trick: In contrast to depending just on the addresses it also stores a cache of the first X characters and the address range in the index to significantly save on system calls and I/O. Since this can be mapped into memory it super-scales to multiple processes accessing the same indexes.
- Each word has a composite address made up from the address (key) of the file it is contained in, the file offset to the start of record (a file can contain multiple records) and the offset from the start of record to the start of the word.
- Each field (container) instance encountered gets added in a separate file its start and end address stored. We use multiple files to better exploit individual file system features, configurations and the potential tuning thereof..
- The vector of the pairs <word, address> is sorted by word. The address column is dumped to disk, the addresses whose words match the first and/or last X characters is dumped as < word fragement, start offset, end offset> where are offsets are into the index above.
- The address of every word is stored and the address range of every field instance is stored.
- An index file tracks the correlation of composite address to file and record.
- Should a path/field/node be of an extended datatype, a handler parses the contents and adds it to a datatype optimised index (date, numerical, geospatial etc.).

What did this deliver?

Unlimited term length, unlimited fields, wildcards and deep search

- For every word we have an address and can open the file and read it. The word could have been encoded. No general need for an intermediate file store (such as JSON). How it was encoded we know from index time and can have classes that are responsible for handling the document format do the right magic.
- Since we know the addresses in all our fields (paths) we can find the paths for any word.
- This lets us, for example, not just search for words in a specific field, a specific path but also for words in the same container without knowing its name or path.
- Since we can read the original file we can also search for unlimited length literals using any wildcard or regex scheme of our dreams—even applied to field names and paths.
- We can even go the other way and ask: what words are in a given field or path instance?
- Since the hierarchical structure of a document is modelled we can walk up (parent) and down (children) from any point in its tree without need to re-parse.
- Since we know all the words and the structure we can even at search reconstitute an alternative encoding without needed to re-parse the original. Its just load and translate using the extracted structure and addresses from the indexing stage.
- Throw in multiple polymorphic datatypes with their own search and ranking algorithms…

Exploiting Structure

(Explicit and Implicit)

The re-Isearch engine exploits document structure, both implicit (XML and other markup) and explicit (visual groupings such as paragraph), to zero in on relevant sections of documents, not just links to documents. These are a heterogeneous mix of text, data (a large number of datatypes including: numerical, computed, range, date, time, geo, boolean etc. as well as a number of hashes including several phonetic), network objects and databases. These datatypes have their own, for their individual datatypes, storage and retrieval algorithms (including relevant ranking and similarity methods).

Project Schmate extends re-Isearch with vector datatypes tuned for embeddings and using structure not just for chunking but for a more optimal passage retrieval.

Why do we need/want “semantic search” — embeddings (dense vectors)?

Are not “keywords” (lexical search) sufficient if not often, in the right hands, superior? And definitely more explainable?

Semantic Search

- **Not everyone is a trained library scientist.** Formulating structured search expressions is not trivial. People often just use a few words and expect magic—avoiding recursive exploration (research).
- **Enhanced user experience**: When search results seem immediately highly relevant, users have a more satisfying experience. They can quickly find the information they need without wading through pages of what they consider irrelevant links.
- Heat maps consistently show users focus on **the top-left portion of a search engine results page (SERP) in an "F-shaped" pattern. This means the vast majority of visual attention—and subsequent clicks—is concentrated on the first few organic results, the top of the page, and any "rich results" (images, featured snippets) that appear there.**
- Lets look at a typical SERP:
  - The first 3–4 results receive the most engagement, with 80% of users clicking within the top four results before doing anything else. The First Result (No. 1) is by far the "hottest" area, often capturing nearly 40% of all clicks.
  - Searches often result in a "zero-click" scenario, where users look at the top snippet and find “their answer” without clicking anything.
- **Projected Improved relevance**: By “functional simulation of *understanding*” the meaning behind a search query, especially complex or ambiguous ones, search engines can deliver the “feeling” of more relevant results. This means users are more likely to feel satisfied with the results on the first try.
- **Increased engagement:** Relevance is key to engagement. When users find what they are looking for, they are more likely to spend time interacting with the content since they more quickly find what they’re looking for.

Large Language Models as Search Ersatz?

- Everyone by now has played around with them, be it Chat-GPT, Claude, Gemini, LLaMA, DeepSeek, … Many see them as an “ersatz” for traditional search engines. But …
- Large language models only “know” what they have been trained on—they hallucinate the rest. Since the world does not stand they would need to be re-trained. This is computationally extremely expensive.
- LLMs are statistical machines and they can track normative use. They are not suited to understanding what I’ve called semantic spheres including social jargon (“to the moon” means something different in Wallstreet Bets than when expressed by Ralf Kramden in the Honeymooners TV show from the 1950s or to Elon Musk’s latest ..). When I traveled in the Soviet Union in the early 1980s I discovered the phrase “fight for peace” meant a belligerent demand for hegemony to the member of the Central Committee I debated (“We are prepared to fight for peace, if you slap us we’re prepared to fight back with machine guns”). Words like “cosmopolitan” were pejorative and typically used to denote Jews.
- The concept of “semantic spheres” refers to the interaction of speakers and listeners. Engaged in conversation they adopt common semantics particular to the “bubble”. Searle, for instance, emphasized that meaning is fundamentally about the speaker's intention to produce a particular effect in the listener, which is achieved through shared conventional rules.
- For the curious: At ISEA2008 (nearly 18 years ago): 14th International Symposium on Electronic Art, for example, we together with the Dutch Design Co-op Metahaven showed how recursive search into spheres can look. The spheres we calculated as clusters in social networks. The idea was that “if people are talking or arguing with one another they will adopt for the conversation some shared semantics”

Large Language Models as Search Ersatz?

- An an aside: the common paradigm of using embeddings for semantic search is not the only semantic model available.
- One can use a domain specific weighted thesaurus and apply term expansion using lexical methods. This thesaurus can be easily built using a domain training set.

```
Import os
from tqdm import tqdm
from sentence_transformers import SentenceTransformer, util
from keybert import KeyBERT
import nltk
import numpy as np

# Download NLTK data
nltk.download('punkt')
nltk.download('stopwords')

# === CONFIGURATION ===
TEXT_FOLDER = "texts"   # Folder containing .txt files
THRESHOLD = 0.65        # Similarity threshold (0.6–0.8 typical)
OUTPUT_FILE = "synonyms.txt"

# === INITIALIZE MODELS ===
print("🔷 Loading models...")
sbert_model = SentenceTransformer('all-MiniLM-L6-v2')
kw_model = KeyBERT(model=sbert_model)
print("✅ Models loaded.")
```

```
# === STEP 1: LOAD TEXT CORPUS ===
texts = []
print(f"\n📂 Loading text files from '{TEXT_FOLDER}'...")
for filename in tqdm(os.listdir(TEXT_FOLDER), desc="Reading files"):
    if filename.endswith(".txt"):
        path = os.path.join(TEXT_FOLDER, filename)
        with open(path, "r", encoding="utf-8") as f:
            texts.append(f.read())

print(f"✅ Loaded {len(texts)} text files.\n")

# === STEP 2: EXTRACT CANDIDATE PHRASES ===
print("🔷 Extracting keyphrases with KeyBERT...")
all_phrases = set()
for text in tqdm(texts, desc="Extracting phrases"):
    keywords = kw_model.extract_keywords(
        text,
        keyphrase_ngram_range=(1, 3),
        stop_words='english',
        top_n=10
    )
    for phrase, _ in keywords:
        all_phrases.add(phrase.lower())

phrases = list(all_phrases)
print(f"✅ Extracted {len(phrases)} unique candidate phrases.\n")
# === STEP 3: EMBED PHRASES ===
print("🔷 Computing embeddings...")
embeddings = sbert_model.encode(phrases, convert_to_tensor=True)
cosine_scores = util.pytorch_cos_sim(embeddings,
embeddings).cpu().numpy()
print("✅ Embeddings computed.\n")

# === STEP 4: BUILD SYNONYM GROUPS WITH PROGRESS ===
print("🔷 Grouping similar phrases...")
synonym_groups = {}
for i, term in enumerate(tqdm(phrases, desc="Finding synonyms")):
    similar_indices = np.where(cosine_scores[i] > THRESHOLD)[0]
    similar_terms = [
        (phrases[j], float(cosine_scores[i][j]))
        for j in similar_indices if j != i
    ]
    if similar_terms:
        sorted_terms = sorted(similar_terms, key=lambda x: -x[1])
        synonym_groups[term] = sorted_terms

print(f"✅ Found {len(synonym_groups)} synonym groups.\n")


# === STEP 5: DEDUPLICATE OVERLAPS ===
print("🔷 Deduplicating overlapping groups...")
final_groups = {}
seen = set()
for main, group in tqdm(synonym_groups.items(), desc="Deduplicating"):
    if main not in seen:
        children = [t for t, _ in group if t not in seen]
        seen.update(children + [main])
        final_groups[main] = group

print(f"✅ Final unique groups: {len(final_groups)}\n")

# === STEP 6: FORMAT OUTPUT ===
print("🔷 Formatting output...")
output_lines = []
for main, group in tqdm(final_groups.items(), desc="Formatting lines"):
    if group:
        children_fmt = [
            f"{term}:{int(score * 100)}" for term, score in group
        ]
        line = f"# {main} = " + "+".join(children_fmt)
        output_lines.append(line)

formatted_output = "\n".join(output_lines)

# === STEP 7: SAVE OUTPUT ===
with open(OUTPUT_FILE, "w", encoding="utf-8") as f:
    f.write(formatted_output)

print(f"\n✅ Synonym list written to '{OUTPUT_FILE}'")
```

Large Language Models as Search Ersatz?

- Corupus confusion given that words can evolve over time to radically shift their semantics (Semantic Drift).
  - Even in the 19th century “awful” meant fearful whose root “awe” denoted a feeling of danger or reverence (John Ruskin in “The Stones of Venice” 1851 argued that the "savageness" of Gothic architecture was a source of awe, representing the untamed, free spirit of the northern builder. Awful = intense reverence). villain referred to a farm hand, clout meant clothing and a “guy” was a grotesque effigy (a reference to Guy Fawkes, a conspirator in the gunpowder plot to blow up the English Parliament in 1605). By the 1830s/1840s in London, it was used to describe anyone wearing bizarre or old-fashioned clothing. Not only is “guy” now in the U.S. any male but Guy Fawkes is increasingly seen as a positive, or at least sympathetic, anti-establishment icon today (see Anonymous),
  - Nice is a great example: Originally from “timid, faint-hearted” (pre-1300); to “fussy, fastidious” (late 14th century); to “dainty, delicate” (circa 1400); to “precise, careful” (in the 1500s, preserved in such terms as a nice distinction and nice and early); and then it came to mean “agreeable, delightful” around 1769); and finally it meant to be “kind, thoughtful” around 1830.
  - “I am sure,” cried Catherine, “I did not mean to say anything wrong; but it is a nice book, and why should I not call it so?” “Very true,” said Henry, “and this is a very nice day, and we are taking a very nice walk; and you are two very nice young ladies. Oh! It is a very nice word indeed! It does for everything.” — Jane Austen’s novel *Northanger Abbey* (1803). Here she is through the character Henry Tilney mocking how the word had become a lazy, all-purpose compliment, losing its original, more precise meaning.
- English shows unusually high semantic instability, especially since 1500.

Large Language Models as Search Ersatz?

- My favourite test set for my indexer is the collected works of Shakespeare. 37 Plays (of the 38 or 39), 912166 words. 22945 unique, 27 characters in the longest word and an average of 7 characters per word. The typical modern English novel, by contrast, has between 5000 and 10000 types (unique words). I'm here excluding works such as Beckett's "Waiting for Godot" which has in its 19K or so words a minimalist vocabulary used to convey monotony and existential absurdity.
- In Shakespeare let us look at a few examples of semantic drift and why even English is problematic.
  - "*That will prove awful both in deed and word.*". This line comes from Shakespeare's Pericles, Prince of Tyre (Act 2, Chorus, line 4), spoken by the narrator, Gower. In Shakespeare's day awful was intended to mean inspiring awe; majestic or fearsome and not the contemporary semantics as something bad or unpleasant. "*Deed and Word*" indicates that Pericles is a virtuous, honorable, and effective leader in both what he does (actions) and what he says. Gower is thus describing Pericles as a "benign lord" (a good/kind ruler), contrasting him with the immoral king in the previous act
  - Contrast that with the Cambridge dictionary where "awful in deed and word" means to be consistently wicked, cruel, or deeply unpleasant in both actions (deeds) and speech (words). It describes a person whose behavior is completely detestable, malicious, or of exceptionally low quality, leaving no aspect of their conduct, whether spoken or acted upon, as decent or acceptable.
  - "*All my fond love thus do I blow to heaven*" — spoken by OTHELLO signifies his definitive abandonment of his love for Desdemona, replacing it with hatred and a commitment to revenge. "Fond" here means foolish, naïve.
  - Let's look at the word "Jealous" as in "And be not jealous on me, gentle Brutus..", "I shall grow jealous of you shortly, Launcelot,..", "My lord, your nobles, jealous of your absence..", "Now, by the jealous queen of heaven, that kiss.." … When Shakespeare wrote of jealousy, he often meant **fear of betrayal**, not envy of someone's possessions.
  - This connects back to the Bible when one speaks of אל קנא A JEALOUS G-D — He is jealous to exact punishment, and does not pass over His rights by pardoning idolatry (Mekhilta) [Rashi].

Large Language Models as Search Ersatz?

- Shakespeare’s writing is heavily structured, particularly in sonnets (iambic pentameter) and blank verse. LLMs have difficulty maintaining strict, long-range, structural constraints over many lines, resulting in a loss of rhyme or meter, as the model's focus shifts from word to word rather than the overall structure.

- LLMs are excellent at surface-level style, but they struggle with metaphor, irony, and deep, nuanced meaning. Shakespeare is heavily reliant on complex metaphor, which requires understanding the deeper, non-literal meaning of words rather than just predicting the next word based on probability.

- Artistic writing requires deliberate, non-obvious choices, which goes against the probabilistic, “average” nature of an LLM's output.

- LLMs often struggle with unique, creative, or less common combinations of words, as they are trained to predict common, high-frequency patterns in data. Artistic language is often unique and non-standard, which can be misread by a model heavily weighted towards common phrases

Large Language Models as Search Ersatz?

- German albeit much more stable than English too has its share of drift. The most common examples are the words “Geil”, “Gift” (originally aligned with the English gift, today it means exclusively poison or drugs as in “Rauschgift”), “Handy” (a mobile phone), Weib/Wib (once denoting any women its become a pejorative), “Toll” (went from crazy or madness as in Tollwut to mean “super”),
- In drift we see the standard patterns of: Narrowing, Broadening, Pejoration and Amelioration
- The internet has profoundly changed language by speeding up evolution, blending formal and informal registers into "netspeak," and introducing new vocabulary. It has popularized abbreviations (LOL, BRB), emojis, and novel slang, while facilitating the rapid spread of global loanwords and creating "algospeak" to bypass content moderation. Based on analysis of training data composition for large language models, the vast majority—often cited as over 60% to 80% or more—of the training data for models like GPT-3 and GPT-4 came from Internet web pages.
- We also have a feedback loop (tail wagging dog):
  - Web pages increasingly created by generative AI tools. Recent estimates suggest that over 50% to 57% of all online content is currently generated or translated by AI. Other more comprehensive studies analyzing newly published content indicate that up to 74% of new web pages contain some level of AI-generated text.
  - As of late 2025, Anthropic also began training on consumer-level conversations (Free, Pro, Max) and coding sessions, unless the user opts out. The same can, of course, be also said for the other players.

Large Language Models as Search Ersatz?

- Latin particularly evolved over centuries but much of Latin's semantic drift happened because it fragmented into new languages. Most of these daughter language stabilized meanings once formed: Romance languages tend to preserve semantic cores.

- **Semantic drift** appears to **correlate** strongly with: **Cultural turnover**, T**echnological innovation**, **Social mobility** and **Contact** with other languages. In this sense many minority languages are quite stable. Modern Icelandic speakers, for example, can still read 800-year-old sagas. Lithuanian has also preserved some very archaic Indo-European features.

Large Language Models as Search Ersatz?

- Statistical issues
  - Words that frequently appear together in training data can trigger a false response regardless of the question's normative meaning.
  - There is also a significant **coverage problem. Modern usage of words vastly outnumbers historical usage in training data. So when a model encounters a word or phrase its strongest statistical associations pull toward the contemporary meaning, which can subtly bias interpretation even when context signals otherwise.**
  - This brings us back to John Searle and his Chinese Room thought experiment.

    PKP

    - Imagine a native English speaker who knows no Chinese locked in a room full of boxes of Chinese symbols (a data base) together with a book of instructions for manipulating the symbols (the program). Imagine that people outside the room send in other Chinese symbols which, unknown to the person in the room, are questions in Chinese (the input). And imagine that by following the instructions in the program the man in the room is able to pass out Chinese symbols which are correct answers to the questions (the output). The program enables the person in the room to pass the Turing Test for understanding Chinese but he does not understand a word of Chinese.
    - The Anthropomorphic Error: Because we, as humans, understand language and possess consciousness, we tend to project these qualities onto systems that mimic our behavior.
    - The Illusion of Understanding: The Chinese Room highlights that an outside observer might "project" intelligence onto a computer simply because the output is convincing. The experiment demonstrates that the appearance of consciousness does not prove consciousness is actually happening.
    - "As-If" Understanding: Searle argues that computer understanding is merely "as-if" understanding, a projection made by humans who see a simulation and mistake it for the real thing.
  - LLMs are primarily System-1: operate by identifying statistical relationships within data, converting text into numerical vectors to predict the most plausible next token with some limited memory (what has been called Type-2). What we are increasingly seeing are inferencing engines (a symbolic, logic-based set of principles) attempting to guide how the neural network learns to evaluate and refine its own responses. This is widely considered a critical, and by many, essential, pathway towards achieving Artificial General Intelligence (AGI).

Large Language Models as Search Ersatz?

- Adversarial Manipulation:
  - **Viktor Klemperer's *LTI - Lingua Tertii Imperii* (1946)** study on the "Language of the Third Reich," argued that Nazi language infiltrated everyday discourse to normalize ideology through the endless repetition of words, acting like "tiny doses of arsenic".
  - Just as the Nazi Party (NSDAP) strategically adopted, adapted, and subverted communist and socialist terminology to appeal to the German working class and diminish the appeal of left-wing parties, and reframe their own far-right ideology as a populist movement, the contemporary MAGA movement has adopted elements of socialist, left-wing, or populist language to appeal to working-class, anti-establishment sentiments, often recontextualizing these phrases to fit a nationalist agenda rather than a collectivist one. This adoption is frequently characterized as "Right-wing populism" or a form of anti-elite rhetoric that mirrors critiques of capitalism, while simultaneously promoting nationalist and protectionist policies.
  - A number of reports have shown how Wikipedia has been subverted by large scale coordinated manipulation of content. Proponents have even its importance beyond Wikipedia given the use in model training.
  - This connects back to the ease with which we've found LLMs can be compromised See my FOSDEM '24 Talk https://archive.fosdem.org/2024/schedule/speaker/9YMVL7/ (Fortify AI against regulation, litigation and lobotomies.
  - According to a joint study from Anthropic, UK AI Security Institute and the Alan Turing Institute it takes as little as 250 documents to poison a LLM of any size. See https://www.anthropic.com/research/small-samples-poison

Large Language Models as Search Ersatz?

- LLMs have poor “memory” beyond the session (and there are often limits here too).
- Most of the foundational LLMs we tend to use have been trained by others. They don’t know “my data”.
- The answer could be the **R**etrieval **A**ugmented **G**eneration paradigm. RAG fuses indexing technology with LLMs.
- RAG allows developers to provide not only the latest research, statistics, or news to the generative models but also to restrict text by attributes such as date— as a side note in EDTF and ISO 8601:2019 Part 2 we created a basis for date ambiguity— or what I termed above as “linguistic sphere".

A little note on dates and handling drift

Since we have no limitations to number of fields or depths of paths we can use explicit structure (markup) to indicate period (date) and search these using either lexical or vector methods.

TEI provides specific attributes for dates that are not precise but since ISO-8601:2019 we can encode them also using the ISO standard which is more powerful than ISO 8601:2004 based structures like

```
<date notBefore="1800" notAfter="1810">Early 19th Century</date>
```

Don’t fully grasp the “vagueness” of the intent in the expression.

19th Century: 19C (using ISO 8601-2 extensions

Semantic Segmentation

We propose to cluster domains and use different embedding models for different general periods, perhaps, of course, with some overlap.

Period boundaries in language are fuzzy — the 17th century bleeds into the 18th, regional variation exists within periods, and individual authors can be archaic or ahead of their time. So retrieval routing logic demands soft boundaries rather than hard cutoffs, perhaps querying adjacent period models and merging results with some weighting scheme.

To create these clusters we propose to exploit a number algorithms to detect semantic drift and work backwards with the source metadata to allow cluster structure to emerge from the language itself.

Algorithms like those based on word2vec temporal slicing, SCAN, or dynamic Bernoulli embeddings are designed to detect *when* and *how* word meanings shift by comparing distributional semantics across time-stamped corpora. The idea is to effectively let the drift signal itself where the meaningful cluster boundaries are, rather than imposing arbitrary period boundaries like "pre-1800" or “Victorian." The advantage of this paradigm is that clusters would be semantically motivated rather than historically arbitrary.

The practical challenge is that drift detection algorithms typically need reasonably dense corpora at each time slice to produce reliable signals. For well-documented periods one can get clean boundaries, but for sparser periods the drift signal gets noisy and the cluster boundaries less trustworthy — which circles back to the data scarcity problem at our core. Though interestingly, noisy boundaries in the drift signal might themselves be informative, suggesting those are precisely the zones where retrieval should query multiple clusters.

An open question, of course, is how to classify an incoming query or document fragment as belonging to a particular period if the user doesn't specify? A lightweight classifier sitting upstream of the embedding step could perhaps work, but it needs to be explored.

Popular Semantic Change Detection Approaches

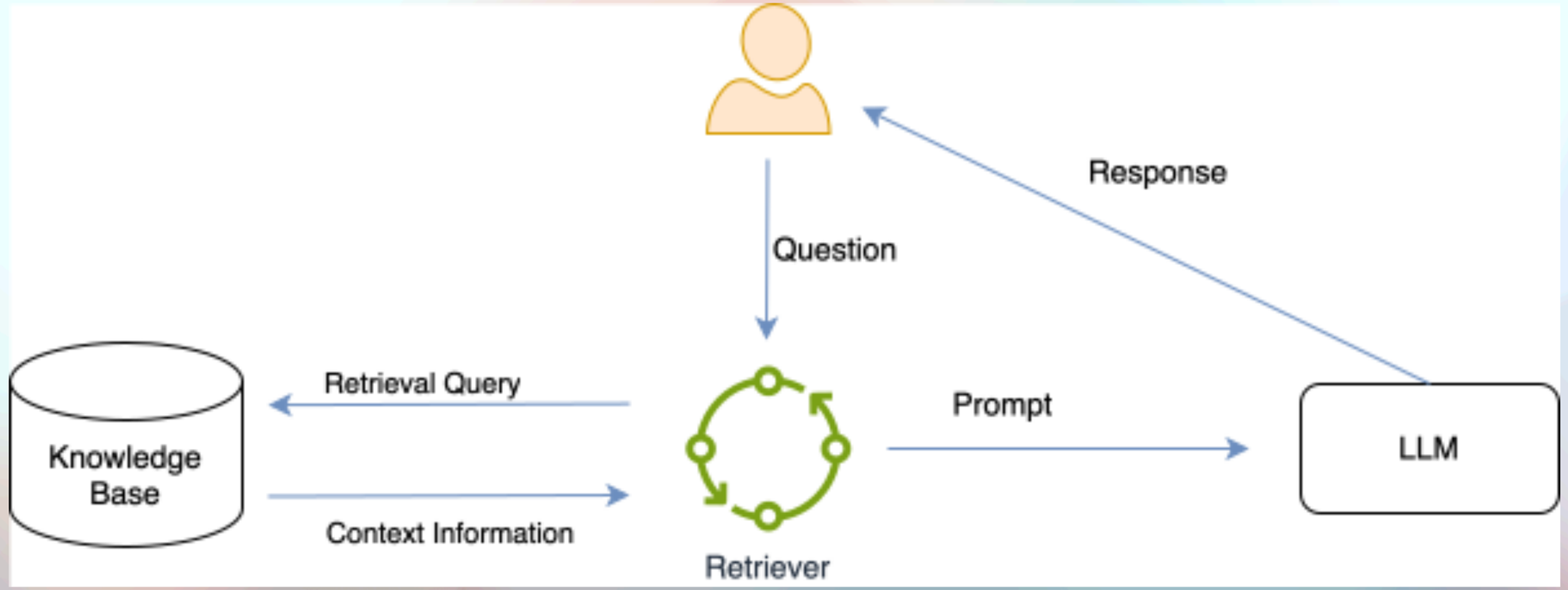
Temporal Random Indexing (TRI):

- It enables the analysis of linguistic, social, or cultural changes by creating multiple semantic spaces for different time periods to detect semantic shift.
- Cheap
- Methodology: The approach typically involves assigning random high-dimensional vectors to words and updating these based on context within specific time windows. See https://github.com/pippokill/tri

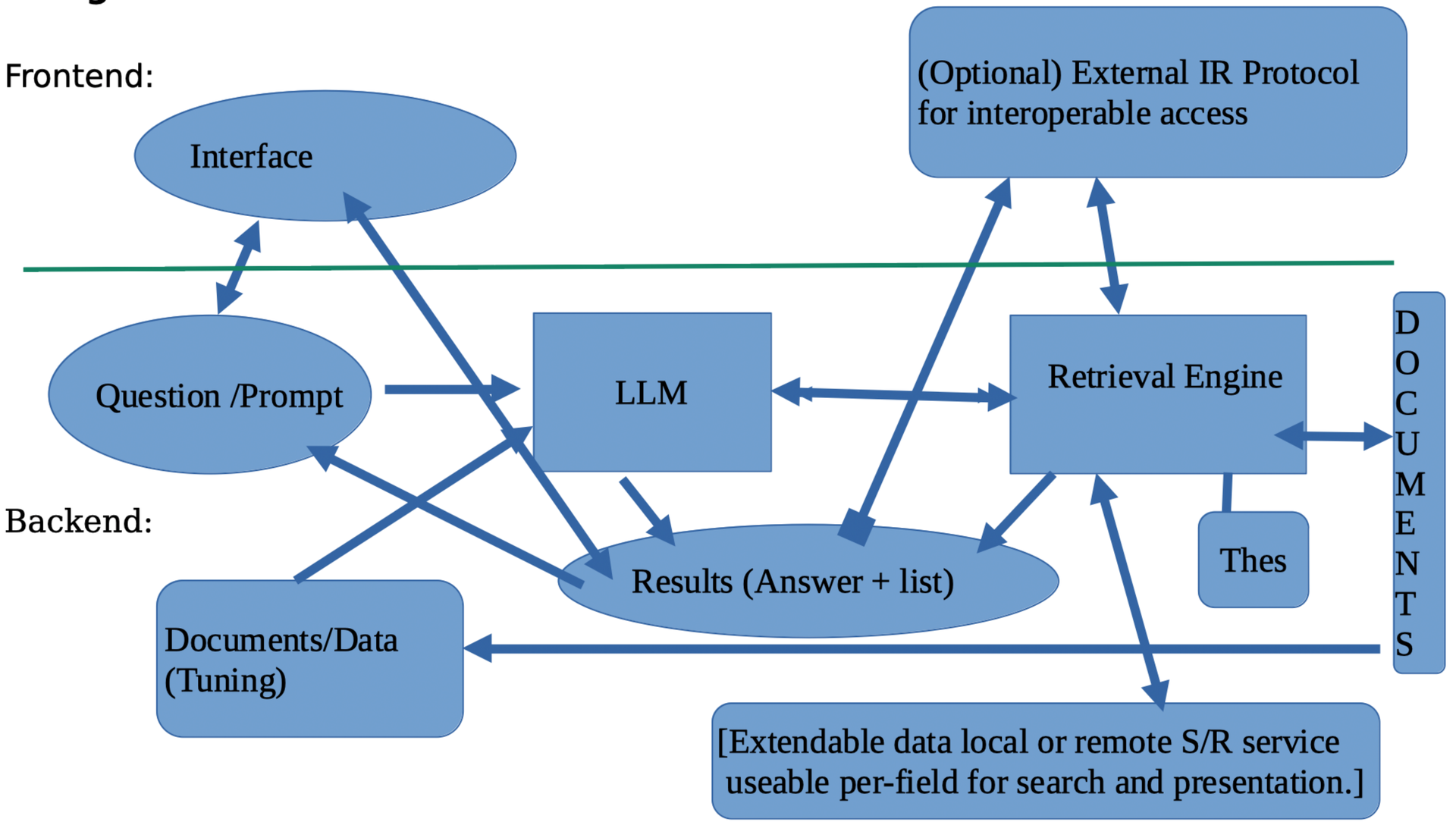
- **SCAF (Semantic Change Analysis Framework):**
  - This approach combines word-embedding types with alignment and frequency, proving superior by outperforming existing approaches in detecting changes by over 40%. See https://github.com/englhardt/scaf
- **Dynamic Word Embeddings (DynamicWE):**
  - Models that learn time-aware embeddings simultaneously and solve the alignment problem at the same time (e.g., using skip-gram with negative sampling over time slices) are generally more robust than purely count-based methods like TRI. See https://github.com/ngillani/dynamic-word-emb
- **Word2vec/Word-Embedding-Based Approaches:**
  - Prediction-based models (like Word2vec trained on yearly slices) often provide superior semantic representation compared to count-based TRI models. Example: https://github.com/mgruppi/schme
- There are a host of others.. like Alignment-Based Methods

The RAG Pipeline

RAG uses search to fortify prompts to deliver information out of scope of the original training of the LLM.



Design



(the optional external IR protocols include, but are not limited to, the lingua franca of libraries: Z39.50/ISO23950 and its Web counterpart OASIS SRU/W)

Since sentence transformers (in contrast to frontier models) are computationally feasible to create for a domain (wiggling our way out of the semantic drift rabbit hole). What's still wrong with RAG then?

Typical RAG challenges or why Re-Isearch+Schmatte for DPR?

While creating a prototype RAG application is these days comparatively easy thanks to sites like LangChain and HuggingFace, making it work well much less performant, robust, or scalable to a large knowledge corpus has proven for many organizations as quite difficult.

1. **Ingest**
    A. Data Extraction
    B. Chunk size and chunking strategy
    C. Robustness and scalability
2. **Search and Retrieval**
3. **Data Security**

Ingest

A) Extraction

Extracting data from diverse types of documents, such as emails, PDFs and office files such as ODF can be challenging. Documents have generally both **explicit** and **implicit** structure (text formats of what is typically called unstructured is not really without structure). The re-lsearch engine already understands these (and more than 80 base types and with these 100s more and with a plugin-in architecture easily extended to support new formats).

Our ODF parser is, for example, the absolute state-of-the-art: the fastest and most accurate (created in coordination with the OASIS ODF TC).

Ingest

B) Chunking

Our approach is variant of context-aware chunking that also takes the hierarchical structure of text into account.

It contains a computationally more modest approach than “agenic” (which uses a LLM to determine chunks) to find better context matches by using the found passages and seeing them in the hierarchical context of the document. In contrast to recursive we view the edge nodes of a document tree as the fine level chunks and determine the optimal passage on the distribution and relation of the retrieved chunks in the tree.

Ingest

C) Robustness and Scalability

In order to be robust and scale one needs a modular and distributed system. The re-Isearch design has been proven over many years in a large number of large scale highly visible deployments.

Beyond the internal document formats (DOCTYPES) there is a mechanism for dynamic loadable plugins. As standard a number of plugins are bundled to provide also templates to generate new ones specific to need.

In the future a plug-in design will also be implemented to extend re-Isearch to support additional ANN (approximate nearest neighbour) algorithms for dense vector retrieval ranking as well as embedding generators.

The 64-bit index (32-bit file IDs) supports a max Input of 16384 Tbytes (Total of all files) and 1 million records per physical index which, in turn, can be aggregated on demand at search-time with up to 254 additional indexes to create a searchable virtual index (max 255 million records). This can be significantly increased by compiling for 64-bit file IDs. In the other direction it can be compiled to produce 32-bit indexes to save on storage.

Search and Retrieval

The power of re-Isearch as a hybrid

a. Because of their chunking size and strategy retrieved passages may sometimes been misaligned to context. Because we support multi-indexes, have a scope aware passage retrieval system-- we retrieve not just the chunks in the similarity top-k but view these in their contextual scope to retrieve a scope that is more inclusive of probable context.

b. This design addresses several well-documented limitations of contemporary retrieval-augmented generation (RAG) models, including misaligned chunking strategies and semantic collapse, the statistical limits of dense retrieval at scale.

c. As corpora grow, embedding distances tend to compress, clusters overlap, and similarity metrics become less reliable indicators of relevance. Rather than treating text as arbitrarily segmented chunks, our approach models text as structured discourse. Since texts are written to be read, they adhere to conventions that convey meaning through syntax and organisation, which in turn provide a grounding for semantics and contextual interpretation.

Search and Retrieval

The power of re-Isearch as a hybrid

d. We also do, among many other things, query augmentation to help contextualize and generate accurate responses, We can also deploy query routing to the most appropriate domain (index or even virtual collection of indexes).

Sustainability

Energy and Computational resource demands.

It's one thing to have a good search but it must also be economically feasible, efficient and sustainable.

While the standard re-Isearch powered node can run on pretty much any platform, even small embedded systems, LLM inferencing (currently) demands a bit more resources.

But instead of costly GPU data-servers (where an NVIDIA GB200 can cost up to $70,000 each in a rack-scale NVL72 system—even the cut-price Asian market 96GB H20 costs $12k— we aim to target accessible platforms.

While companies such as Google sing praise about their massive context length (large context or LC) models they are heavily data-center based and energy intensive. By constraining computational requirement we can leverage existing hardware and adhere to our aim to eliminate the need for costly data centers and reduce the environmental impact for search.

Just as with the standard version, it is intended that users will be able to host their own data under their own well defined conditions.

Power Constrained Platforms

Energy and Computational resource demands.

Our main development platforms (and initial reference platforms) are Apple silicon (MacBook M), NVIDIA AGX Orin 64 and assorted NVIDIA GPU powered PCs running Ubuntu Linux.

Our use of GPUs/NPUs is limited to the text vectorisation (Transformer Model). Model selection defines, more or less, the hardware demands. Vector search is purely CPU (and RAM) as our algorithms are “fast enough” (with 768 dimensional Fp32 vectors over 3000 per thread QPS on an Apple M1 and with multiple shards effectively as high as 13K QPS).

With the recent introduction of the $250 (the 8GB version currently available for around €275 Netto in Europe) NVIDIA Nano Super (basically a 8GB trimmed Orin NX but still delivering about 40-80 TOPs and 68 to 102 GB/s bandwidth) a low cost edge platform is already available suitable for many highly quantised models. The 16GB Apple M4 Mini (€500) has similar performance at 38 TOPs (and 100 GB/s) but with more memory—the M5 (mid-2026) will be similarly priced but offer at least 20% effective better overall performance. Another low-cost solution is also posed by the Raspberry Pi5 + AI Hat 2. The “Hat” provides 8 GB LPDDR4X-4267 SDRAM and a Hailo-10H accelerator (40 TOPS INT4) for around €115 Netto— in addition to the cost of the Pi5 (the 16GB is ~€165), case and storage.

Other significantly more powerful (faster and more VRAM) I but still moderately priced edge platforms are also available. The 1000 TOPs 128GB 6144 CUDA core NVIDIA GB10 (there are several including, of course, the DGX Spark) is around €4000. With 128GB there is also the € 3000 NVIDIA AGX Thor (273 Gb/s, 2560 CUDA cores) .

Both hardware and software at not standing still. We expect suitable (inferencing) hardware to become rather pedestrian over the next few years—the main bottleneck right now is memory availability (a product of hyper scalar mania).

Flexibility and Future Forward

Search must be economically feasible, efficient and sustainable.

We have opted for a flexible architectural design to support multiple vector stores including, of course, HNSW. The basic idea is to view them as data types and within their handlers pipelines to selectable/extendable embeddings models.

We have found, for example, that some algorithms work for some data really well but for others sub-optimally and visa-versa. Another aspect of our design, however, is to enable a simple pipeline where documents handed over to the ingest get processed by the appropriate document handler and data types detected, structure exploited for chunks, embeddings created and stored in an appropriate vector data store.

We currently use for embeddings both bert.cpp and llama.cpp as BERT support was added to llama.cpp in February 2024 via PR #5423 GitHub. Unfortunately llama.cpp has since dropped support for the ggml file format.

Schmate v1. Indexes

Semantic search with SBERT + GGML/GGUF + HNSWlib.

This system provides a sentence-embedding search engine built on:

- SBERT (Sentence-BERT) model running via ggml (no dependency on frameworks like PyTorch or Tensorflow).
- Loadable transformer models using the GGUF and GGML file formats—quite common on HuggingFace.
- Hierarchical Navigable Small World (HNSW) algorithm for fast approximate nearest neighbor (ANN) retrieval
- We use a significantly enhanced (adding among other features quantized spaces) and turbo-charged (including support for x86 and ARM SIMD) HNSWlib for approximate nearest-neighbor search, and efficient mmap-backed offset storage for text retrieval. The system supports sharded (multi-threaded) HNSW indices, multiple search modes (kNN, radius, relative, adaptive, epsilon), deletion/undelete, merges, and incremental on-disk flushing.
- Memory-mapped offset files for persistent, efficient text–embedding linkage
- Optional re-scoring for quantised embeddings (such as binary) via an LSM vector store that supports: in-memory and memory mapped access.
- LRU caching of indexes using available RAM to prevent both swapping and OOM when writing to a number of different indexes.
- We also include training for hyperparameter optimization.

HNSWlib Hyperparamter Optimisation

Hyperparameter Tuner using run-time adaptive adjustments.

- EfSearchTuner: determine a better choice for ef.

| ------------------------------ | --------- | --- |
🔷 **Speed-critical (real-time)**	`16–64`	Fast, but may miss some near neighbors.
🔷 **Balanced (default)**	`100–200`	Common sweet spot (used by FAISS, sentence-transformers).
🔷 **High recall / offline batch**	`300–800`	High accuracy, slower query.
🔷 **Max recall / evaluation**	`1000+`	Nearly exact results, but 10× slower.

- EpsilonTuner: determine a better choice for epsilon in epsilon search.

Situation	Behavior
Too few results (<80% of target)	Increase ε slightly (expand radius).
Too many results (>120% of target)	Shrink ε slightly (tighten radius).
Stable result density	ε converges.

Performance Improvements

Turbo-charging the already lightning fast HNSWlib with a large number of SIMD optimisations.

```
L2 Distance: 3-5x faster than scalar
Inner Product: 3-5x faster than scalar
Apple Silicon (M1/M2/M3): 4-6x faster
AWS Graviton: 3-4x faster

Performance Boost using O3 and Neon (NEON) over straight
compile (Scalar): 20x

Platform: Apple M1 Pro
clang version 20.1.7
Darwin Kernel Version 24.6.0

L2 Distance (768 dimensions, 100000 iterations):
  Scalar: 2796.45 ns
  NEON:   139.22 ns
  Speedup: 20x

Inner Product (768 dimensions, 100000 iterations):
  Scalar: 2636.55 ns
  NEON:   132.53 ns
  Speedup: 20x

The performance improvement with the new NEON code
is on the order of 4.5x (Inner Product) -  6x (L2)

L2                Throughput: 841 vectors/sec.      QPS: 3036 queries/sec.  QPS: 10484 queries/sec (MT)
Inner Product     Throughput: 869 vectors/sec       QPS: 3049 queries/sec.  QPS: 13760 queries/sec (MT)
```

SIMD Platforms

Our main focus for SIMD is Intel/AMD x86-64 platforms and ARM 64bit.

-SSE (Streaming SIMD Extensions using 128-bit registers and floating-point operations)
-AVX2 (Advanced Vector Extensions using 256-bit registers)
-AVX512 (Advanced Vector Extensions 512)

-ARM NEON (Standard on nearly all ARM processors. It is a mandatory feature for Android devices running version 6.0 or higher)
-ARM SVE (Scalable Vector Extensions).

While NEON is common SVE is currently available only with Fujitsu A64FX, AWS Graviton3 and Graviton4, Alibaba Yitian 710, Google Axion, and the ARM Cortex-A710 and Neoverse V1 cores. In particular not on Apple M-silicon at this time.

Quantization

Quantization is used to dramatically reduce memory usage and storage requirements, making it more scalable, especially for large datasets.

This system provides alongside FLOAT32, Quantization to Binary (1-bit), Ternary (1.58 bit), Nibble (4-bit) and Octet (8-bit). It also supports (not really quantisation but packing) FP16 and BF16.

Quantisation is realised through a selection of algorithms: Standard, Better, Centroid, Rotational, RaBitQ and RabitQ-Extended.

Many models are already pre-quantised so we also implemented a “pass-through” (PASS) to enable native support of 1,2,3,4,5,6,8 and 16-bit integer and Bf16 and Fp16 pre-quantised models. This is useful for **small models** like all-MiniLM-L6-v2 with 4-bit quantization (which is only 14MB).

- We also support optional re-scoring. It gives you speed of quantized search with accuracy close to full-precision!
- For binary we’ve found RaBitQ to be currently the best:
- PASS: Since many GGML/GGUF models are already quantised we allow for pass-through quantisation, e.g. the vectors are decoded and packed into their size. A 4-bit model, for instance remains internally as 4-bit.

Quantization Performance

⭐ Use BIN1-RABITQ for production

```
Dimension: 384    Data points: 50000    Queries: 1000
Params: M = 32, ef_construction = 400, ef_search = 400, k = 10

Mode                   Build (ms)  Query (ms)    Recall Bytes/Vec Memory (KB)
------------------------------------------------------------------------------
BASELINE-L2 (FLOAT32)    23317                   0.9864
INT8-STANDARD            18228.4      363.46    0.9885       384        18750
INT8-CENTROID            18483.1      375.71    0.9864       384        18750
INT8-ROTATIONAL          19952.2      413.05    0.9872       384        18750
INT4-STANDARD            77728.5     1359.43    0.9903       192         9375
INT4-ROTATIONAL          81096.9     1368.87    0.9859       192         9375
BIN1-STANDARD             4884.9       83.97    0.8504        48         2343
BIN1-BETTER               3380.7       49.37    0.6087        48         2343
BIN1-CENTROID             4807.9       81.08    0.8205        48         2343
BIN1-ROTATIONAL           8913.5      134.93    0.8452        48         2343
BIN1-RABITQ               4324.5       76.55    0.9999       112         5468
BIN1-RABITQ-EXT           7041.7      167.17    0.9987       304        14843

=== Analysis ===
Best recall: BIN1-RABITQ (0.9999)
Fastest query: BIN1-BETTER (49.37 ms)
Smallest memory: BIN1-STANDARD (2343 KB)
```

Development Organisation

Based in Munich in association with ExoDAO Network Association (Zurich)

- ExoDAO is a Swiss Association for the Public Good established in 2022 in Zurich, Switzerland by Ole Mueller, Edward Zimmermann and Yoel Zimmermann around the Student Project House (SPH) of the Swiss Federal Institute of Technology in Zurich (ETH-Z). It has been tasked with a “moonshot” to develop a next generation Internet search. Its founding was catalysed by a grant from the Open Data Switzerland/Mercator Foundation Switzerland. The team is currently divided between Zurich (CH), Cambridge USA (Harvard) and Munich.

- Through the founding of a re-Isearch org the development of re-Isearch is disentangled from ExoDAO protocol development. It is intended as a clear signal that one can use re-Isearch without participating in the ExoDAO Network.

- Project Schmate provides code that may also be used outside of re-Isearch. This also de-tangles it as it has many other uses as a vector store. Since its C++ it can easily be embedded into a number of other environments such as Python.

- Project Language Machines builds upon the groundwork laid out in Schmate adding the LLM layer. It is currently seeking public funding.

Longer Term Development

Pipeline for LLM training.

- Reference: The LRZ (the supercomputing centre of the Bavarian Academy of Sciences) Llama-GENBA-10B.
- This trilingual language model is based on Meta's Large Language Model (LLM) Llama, version 3.1-8B, and was trained by researchers at LRZ and Cerebras Systems with 10 billion parameters using a dataset of 164 billion tokens. Llama-GENBA-10B is an inclusive and resource-efficient base model that not only translates but also generates texts in English, German and Bavarian.
- The Bavarian corpus consisted of merely 20M tokens upsampled to 80M with the English and German datasets each truncated to 82B tokens.
- To train Llama GENBA 10B, the CS2 system consumed around 35 megawatt hours of energy in 66 days.
- See https://arxiv.org/pdf/2509.05668
- By comparison: there were an estimated between 50M (lowest estimate) and **339.1M (highest) Latin tokens** in the GPT-3 training data. The Common Corpus (2024/2025) contains 2.267 trillion tokens of which according to https://arxiv.org/abs/2506.01732 (“Common Corpus: The Largest Collection of Ethical Data for LLM Pre-Training”, Langlais et al) there were an estimated 34B Latin tokens— 27.2B from books. We have no current estimate on the number of Latin tokens in the HPLT 3.0 (2025) release. In contrast to Bavarian, Latin covers a very long and diverse history, whence significant semantic drift. Most tokens are Neo-Latin given that it was the dominant language for intellectual, scientific, and scholarly work in Europe for centuries. Despite the aims of Neo-Latin, as an artificial language, to adhere to standardized semantics, it still experienced drift through the necessity of adapting to new developments and cultural shifts. As our focus is semantics we will need to segment into overlapping clusters.
- Train or fine-tune period and domain-specific embedding models on the resulting clusters from drift detection, ensuring that semantic geometry reflects the language norms of each cluster rather than modern usage.
- For Finnish we have the Viking7B model trained on Finnish, English, Swedish, Danish, Norwegian, Icelandic. It is important to note that Viking 7B is a base model and not quite yet suitable for Q/A conversational use.
- These days a lot of people are building on Qwen series of models for fine-tuning given its multilingual tokenization. Qwen's tokenization is considered superior primarily because it adopts a byte-level Byte Pair Encoding (BPE) approach with a large, diverse vocabulary that eliminates unknown words (OOV). Llama uses BPE but it focuses on speed and English efficiency rather than multilingual performance (especially Asian),
- Right now the top of the class 10B models (7-12B) that include Latin are: Qwen2.5-7B, Mistral 7B and Llama-3.2-11B (including Hermes). The best results appear to be fine-tuned with **LoRA** (Low-Rank Adaptation) on Classical Latin datasets. Have not seen DORA (Weight-Decomposed Low-Rank Adaptation) used here yet…
- For model tuning we plan on using QLoRA-PEFT (experimenting also with QALoRa) tuning given its smaller memory footprint.

Main Repository

https://github.com/re-lsearch/Schmate

Schmate project was funded through the NGI0 Commons Fund, a fund established by NLnet with financial support from the European Commission's Next Generation Internet programme, under the aegis of DG Communications Networks, Content and Technology under grant agreement Nº 101135429. Additional funding is made available by the Swiss State Secretariat for Education, Research and Innovation (SERI).